Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Specializarea: Electronică Aplicată | Grupa: 444B

Documentație Proiect Tehnologii Web

Sistem de Crawling, Bază de Date și Interfață Web pentru Teatrul Davila

Student:
Petronela-Georgia-Paula CIULACU
Alina-Florentina CIOACĂ
Anul de studiu: 2026 – 2027

Cuprins

1 Introducere – Prezentarea temei **3**

2 Considerente teoretice și tehnologii folosite **3**

2.1 Crawling 3

2.2 Scraping 4

2.3 Parsing 4

2.4 Baza de date 5

2.5 Frontend și Backend unificat 5

3 Utilizarea instrumentelor AI **6**

3.1 Depanarea stărilor TCP ale rețelei 6

3.2 Soluționarea erorii 404 în arhitectura SPA și unificarea serverelor 7

3.3 Arhitectura de tip Multi-Threading pentru execuția concurentă 7

3.4 Integrarea componentelor: Tranziția de la date statice la API real 8

3.5 Implementarea rutelor și a logicii SPA conform specificațiilor stricte 9

4 Instalare și rulare **9**

4.1 Cerințe preliminare 9

4.2 Instalarea dependențelor 10

4.3 Rularea și oprirea aplicației 10

4.4 Utilizarea interfeței web 11

5 Concluzii și perspective de dezvoltare **13**

5.1 Perspective de dezvoltare 13

6 Bibliografie **14**

1 Introducere – Prezentarea temei

Proiectul de față își propune dezvoltarea unei soluții software integrate, menite să automatizeze procesul de preluare, stocare și afișare a programului și repertoriului Teatrului Davila. Obiectivul principal este eliminarea necesității de a verifica manual modificările survenite pe platforma oficială a teatrului, oferind în același timp utilizatorilor o interfață web modernă, rapidă și responsivă.

Din punct de vedere structural, arhitectura sistemului este de tip client-server și a fost divizată în trei componente majore care funcționează sincronizat:

- Un modul autonom de tip *crawler/scraper* care extrage ciclic informațiile și afișele de pe site-ul oficial.
- O bază de date relațională MySQL utilizată pentru stocarea și interogarea eficientă a spectacolelor și a calendarului de reprezentații.
- Un server web unificat bazat pe arhitectură REST, responsabil de servirea datelor către o interfață cu utilizatorul implementată sub forma unui *Single Page Application*.

Această abordare tehnică garantează o navigare fluidă, fără reîncărcarea completă a paginilor la fiecare acțiune, asigurând o experiență de utilizare optimizată atât pe sisteme desktop, cât și pe terminale mobile.

2 Considerente teoretice și tehnologii folosite

Arhitectura sistemului implementat se bazează pe o abordare hibridă, modulară, care separă logic procesele de achiziție a datelor de cele de stocare și prezentare. Întregul ecosistem combină limbajul **Python**, recunoscut pentru robustețea bibliotecilor sale de automatizare și servere web, cu **JavaScript** , HTML5 și CSS3 pentru o experiență client interactivă. În cele ce urmează, sunt detaliate conceptele teoretice care au stat la baza realizării proiectului.

2.1 Crawling

Crawling-ul web reprezintă procesul automatizat de explorare sistematică a internetului sau a unei rețele de adrese web, cu scopul de a descoperi, indexa și actualiza conținut. Din punct de vedere teoretic, un crawler funcționează prin inițierea unor conexiuni de rețea și emiterea de cereri HTTP GET către serverele țintă.

În cadrul acestui proiect, crawler-ul dezvoltat este unul de tip *focused crawler*, deoarece nu scanează internetul la întâmplare, ci are o traiectorie precisă: rutele site-ului oficial

al Teatrului Davila. Implementarea sa în Python implică rularea într-un fir de execuție separat, pentru a funcționa asincron în raport cu serverul web.

Pentru a respecta etica accesării automate și a preveni blocarea cererilor precum mecanisme de tip anti-DDoS, crawler-ul utilizează:

- **Headere HTTP modificate:** Se folosește un `User-Agent` valid pentru a simula un browser legitim.
- **Temporizare:** Implementarea unei bucle infinite cu instrucțiuni de tip *sleep* pentru a interoga site-ul doar la intervale regulate de timp, minimizând încărcarea serverului sursă.

2.2 Scraping

Spre deosebire de crawling, care gestionează navigarea, scraping-ul definește acțiunea de extragere țintită a datelor din conținutul brut descărcat. Paginile web sunt redactate în limbajul de marcare HTML, care este reprezentat în memorie sub forma unei structuri arborescente complexe, denumită **DOM**.

Scraping-ul presupune parsarea acestui arbore DOM și utilizarea de selectori specifici, precum selectori CSS, pentru a identifica și izola elementele de interes, ignorând codul de stilizare sau funcționalitățile irelevante. În dezvoltarea acestui modul, s-au implementat mecanisme capabile să extragă elemente textuale și elemente media.

O componentă critică a modulului de scraping din acest proiect a fost gestionarea fișierelor binare. La identificarea unui afiș, scraper-ul extrage adresa URL a imaginii, inițiază un nou transfer binar de date și salvează fișierul fizic în sistemul local. Această decizie arhitecturală de stocare locală garantează că aplicația dezvoltată va putea afișa posterele chiar dacă imaginile originale sunt șterse de pe site-ul sursă.

2.3 Parsing

În știința calculatoarelor, parsing-ul este procesul de analiză a unui șir de simboluri pentru a determina structura sa gramaticală cu privire la un limbaj formal dat. În contextul prelucrării datelor web, parsing-ul este esențial pentru normalizarea și curățarea datelor neprelucrate obținute în etapa de scraping.

Datele extrase din paginile teatrului sunt adesea nestructurate sau formatate inconsecvent. Modulul de parsing asigură:

- **Conversia tipurilor de date:** Transformarea textelor ce reprezintă date și ore în obiecte de tip `datetime` pentru a putea fi ordonate cronologic și stocate corect în baza de date.

- **Serializarea JSON :** Elementele complexe și variabile ca dimensiune, precum distribuția sau echipa artistică, nu pot fi stocate eficient în coloane clasice. Ele sunt transformate în dicționare și serializate sub formă de șiruri JSON.

- **Deserializarea:** La momentul solicitării datelor de către frontend, scriptul JavaScript execută o parsare inversă, reconstruind colecțiile de date în obiecte native pentru manipulare și afișare dinamică.

2.4 Baza de date

Pentru persistența datelor a fost ales un model relațional, implementat folosind **MySQL**. Bazele de date relaționale asigură integritatea informațiilor prin respectarea principiilor ACID și prin utilizarea cheilor primare și externe.

Structura logică este optimizată și normalizată, fiind compusă din două entități principale interconectate printr-o relație de tip *One-to-Many* :

1. **Tabela `shows`:** Stochează proprietățile intrinseci și invariabile ale spectacolelor .

2. **Tabela `performances`:** Gestionează instanțele temporale , conținând un ID propriu, data și ora de start, starea biletelor, locația/sala și cheia externă `show_id`, care face referire la spectacolul aferent din tabela `shows`.

Comunicarea dintre backend-ul Python și serverul MySQL s-a realizat folosind un conector nativ `pymysql`. Pentru a facilita transmiterea datelor către frontend, s-a utilizat un cursor de tip dicționar `DictCursor`. De asemenea, datele sunt introduse în sistem folosind interogări parametrizate, o măsură fundamentală de securitate pentru a preveni vulnerabilitățile de tip SQL Injection.

2.5 Frontend și Backend unificat

Interfața vizuală a proiectului este construită după paradigma **Single Page Application** . Spre deosebire de arhitecturile tradiționale MPA, unde fiecare acțiune a utilizatorului forțează browser-ul să solicite un nou document HTML și să reîncarce complet ecranul, o aplicație SPA încarcă un singur document.

Toate actualizările vizuale ulterioare sunt realizate asincron. Pentru a implementa această tehnologie folosind Vanilla JavaScript, s-au utilizat următoarele concepte:

- **DOM Manipulation:** Generarea sau distrugerea elementelor HTML exclusiv din codul JavaScript.

- **Fetch API:** Inițierea cererilor asincrone de rețea bazate pe Promises, pentru a prelua date din baza de date în fundal.

- **History API:** Utilizarea metodelor `pushState()` și capturarea evenimentelor `popstate` pentru a schimba URL-ul din bara browser-ului și a permite navigarea, fără a genera o cerere HTTP reală către server.

O provocare tehnică specifică arhitecturii SPA este comportamentul la reîncărcarea forțată pe o rută virtuală. Deoarece rutele precum `/shows/1` nu există ca fișiere fizice pe disc, serverele statice răspund cu eroarea *404 Not Found*.

Pentru a soluționa acest aspect și pentru a oferi acces securizat la baza de date, a fost implementat un **Backend unificat** bazat pe micro-framework-ul **Flask** . Serverul Flask rutează cererile pe un singur port și îndeplinește un rol dual:

1. **API RESTful:** Răspunde la rute specifice de tipul `/api/shows` cu date interogate în timp real din MySQL, formatate ca JSON.

2. **Server de fișiere statice :** Gestionează un mecanism prin care, indiferent de calea solicitată de browser , serverul returnează întotdeauna fișierul principal `index.html`. Ulterior, logica de rutare internă din JavaScript preia controlul și decide ce conținut trebuie afișat.

3 Utilizarea instrumentelor AI

Pe parcursul dezvoltării acestui proiect, modelul conversațional de inteligență artificială a fost utilizat în rol de asistent de programare, consultant pentru decizii arhitecturale și instrument avansat de debugging. Interacțiunea cu AI-ul a accelerat rezolvarea problemelor tehnice complexe, în special cele legate de stările rețelei și arhitectura serverului.

Mai jos sunt detaliate 5 situații critice în care asistența AI a fost determinantă, incluzând prompturile utilizate, sinteza răspunsurilor primite și o analiză critică a soluțiilor propuse.

3.1 Depanarea stărilor TCP ale rețelei

Problema întâmpinată: În timpul dezvoltării și testării interfeței web pe un server local static , s-a observat că, chiar și după oprirea procesului serverului din terminal, portul rămânea ocupat. Comanda `netstat -ano` returna zeci de conexiuni active, generând suspiciunea unei infectări cu malware sau a unei erori grave de sistem.

Prompt utilizat:

> *„de ce am atat de multe conexiuni pe 8000 cu netstat -ano? deci e serverul inchis, mie imi apar zeci de conexiuni pe 8000. bai frate tu ai inceput sa nu ma mai ajuti deloc, intrebarea mea e de ce, in general, fara nicio legatura cu acest proiect, am atatea conexiuni pe portul 8000, sunt virusată ce sunt??????????????”*

Răspunsul AI : AI-ul a infirmat imediat ipoteza infectării cu viruși și a explicat comportamentul la nivel de sistem de operare. A clarificat că browser-ul deschide simultan mai multe conexiuni pentru a descărca resurse în paralel . După finalizarea descărcării, sistemul de operare Windows păstrează acele *socket*-uri în starea `TIME_WAIT` timp de 1-2 minute . Zecile de conexiuni vizibile erau doar „fantome" ale refresh-urilor anterioare, având PID-ul 0 și urmau să dispară automat.

Analiza critică a soluției: Explicația a fost extrem de valoroasă. A corectat o confuzie legată de securitatea sistemului, transferând problema din zona de *malware* în zona de înțelegere profundă a stivei TCP/IP și a modului în care sistemele de operare gestionează închiderea conexiunilor de rețea. Soluția a prevenit acțiuni drastice inutile.

3.2 Soluționarea erorii 404 în arhitectura SPA și unificarea serverelor

Problema întâmpinată: Interfața web naviga corect între pagini folosind JavaScript, dar la o reîncărcare manuală pe orice pagină în afară de rădăcină, serverul Python de frontend returna eroarea *404 Not Found*. Mai mult, sistemul necesita rularea a două servere distincte: unul pentru frontend și unul pentru backend/API .

Prompt utilizat:

„de ce daca dau refresh pe orice pagina inafara de home, imi da 404"

Răspunsul AI : Inteligența artificială a identificat imediat particularitatea arhitecturii SPA: rutele afișate în browser sunt virtuale, gestionate de `History API`, dar la *Refresh*, browser-ul cere fizic acel fișier de la server. Deoarece fișierul nu există, apare eroarea 404. Pentru a rezolva problema elegant, AI-ul a propus **renunțarea la serverul de frontend de pe portul 8000** și unificarea întregii aplicații sub serverul Flask pe portul 5000. A generat codul pentru o rută *catch-all* în Flask care, indiferent de calea solicitată, returnează `index.html`, lăsând ulterior JavaScript-ul să randeze view-ul corect.

Analiza critică a soluției: Aceasta a fost cea mai importantă decizie arhitecturală sugerată de AI. Unificarea serverelor nu doar că a rezolvat bug-ul de routing, dar a simplificat enorm fluxul de lucru. În plus, prin rularea frontend-ului și backend-ului pe același domeniu și port, a fost eliminată necesitatea configurării politicilor de securitate CORS, sporind performanța și claritatea codului.

3.3 Arhitectura de tip Multi-Threading pentru execuția concurentă

Problema întâmpinată: Sistemul final trebuia pornit dintr-un singur punct de intrare. Cu toate acestea, crawler-ul rulează într-o buclă infinită, iar serverul Flask folosește o funcție blocantă. Apelarea lor secvențială ar fi dus la blocarea unuia dintre procese.

Prompt utilizat:

„deci eu planuiesc sa fac un fisier main.py care porneste si serverul si crawlerul, cum le pornesc si cum manageuiesc inchiderea acestora"

Răspunsul AI : Asistentul AI a generat un script unificator bazat pe modulul `threading` din Python. A propus instanțierea a două fire de execuție separate cu parametrul `daemon=True`, unul pentru serverul web și celălalt pentru crawler. De asemenea, a inclus un bloc `try...except KeyboardInterrupt` în firul principal pentru a capta semnalul `Ctrl+C` și a opri ambele procese în siguranță.

Analiza critică a soluției: Implementarea multi-threading a oferit aplicației un comportament la standarde profesionale de *deployment*. Folosirea *daemon threads* a asigurat că procesele de fundal nu rămân blocate în sistem după închiderea scriptului principal, oferind un control precis și o experiență de utilizare unitară. AI-ul a demonstrat capacitatea de a anticipa problemele de concurență și blocare a memoriei, furnizând o soluție robustă și ușor de testat.

3.4 Integrarea componentelor: Tranziția de la date statice la API real

Problema întâmpinată: În faza inițială de dezvoltare, interfața web fusese testată folosind date statice, definite direct în cod. A apărut necesitatea conceptuală și practică de a lega această interfață de baza de date MySQL populată de crawler.

Prompt utilizat:

„acum ca am lamurit asta, hai sa o luam pas cu pas. Eu am un front end si un back end. backend e bassically in momentul de fata baza de date. Cum fac sa le leg intre ele"

Răspunsul AI : Inteligența artificială a explicat conceptual rolul unui server web ca „translator" între browser și baza de date. A furnizat codul necesar pentru crearea rutelor în framework-ul Flask și a explicat mecanismul prin care scriptul `app.js` trebuie să folosească funcția nativă `fetch()` asincronă pentru a cere datele sub formă de JSON, înlocuind array-urile statice.

Analiza critică a soluției: Acest pas a fost fundamental pentru transformarea proiectului dintr-un simplu prototip vizual într-o aplicație dinamică, *data-driven*. Răspunsul AI a ajutat la clarificarea fluxului de date , facilitând scrierea unui cod decuplat și scalabil.

3.5 Implementarea rutelor și a logicii SPA conform specificațiilor stricte

Problema întâmpinată: Proiectul impunea o structură foarte strictă a rutelor și afișarea unor informații specifice preluate din structuri JSON , toate acestea trebuind implementate *Vanilla*.

Prompt utilizat:

> *„Repet, eu asta am nevoie sa am in site: Frontend Proiectul va include obligatoriu o interfață web responsive. Rutele aplicației vor folosi denumiri în limba engleză. / → spectacole și reprezentații viitoare [...] /shows/{id} → afiș, titlu, autor / regizor, descriere, durată, recomandare de vârstă, distribuție, echipă artistică [...]"*

Răspunsul AI : Plecând de la cerințele stricte furnizate, AI-ul a refactorizat complet scriptul `app.js`. A implementat un ruter custom în JavaScript bazat pe expresii regulate capabil să citească variabile din URL. De asemenea, a furnizat funcții helper capabile să deserializeze corect câmpurile complexe din baza de date și să le randeze dinamic sub formă de liste HTML.

Analiza critică a soluției: Promptarea cu specificațiile exacte ale proiectului a rezultat într-o generare de cod foarte precisă. AI-ul a preluat complexitatea manipulării DOM-ului și a rutării din JavaScript pur, economisind zeci de ore de dezvoltare și validare sintactică. Soluția a respectat întocmai cerințele academice, integrând eficient și un filtru de căutare în timp real aplicat direct pe datele preluate.

4 Instalare și rulare

Aplicația a fost proiectată pentru a fi ușor de configurat și lansat într-un mediu local de dezvoltare sau producție, având un punct unic de execuție care orchestrează atât colectarea datelor, cât și serverul web.

4.1 Cerințe preliminare

Pentru a rula proiectul, sistemul gazdă trebuie să îndeplinească următoarele condiții:

- **Python 3.8 sau mai nou** instalat și adăugat în variabila de mediu `PATH`.
- **Server MySQL** activ , având creată baza de date și tabelele aferente.
- **Fișierul de configurare a credențialelor:** Un fișier text securizat în afara repository-ului public, care conține pe linii separate parametrii de conectare la baza de date.

4.2 Instalarea dependențelor

Se deschide un terminal în directorul rădăcină al proiectului și se instalează bibliotecile Python necesare folosind managerul de pachete `pip`:

```
pip install flask pymysql requests beautifulsoup4
```

Structura directoarelor proiectului trebuie să fie organizată astfel încât serverul Flask să poată accesa atât fișierele statice ale interfeței, cât și posterele descărcate local de crawler .

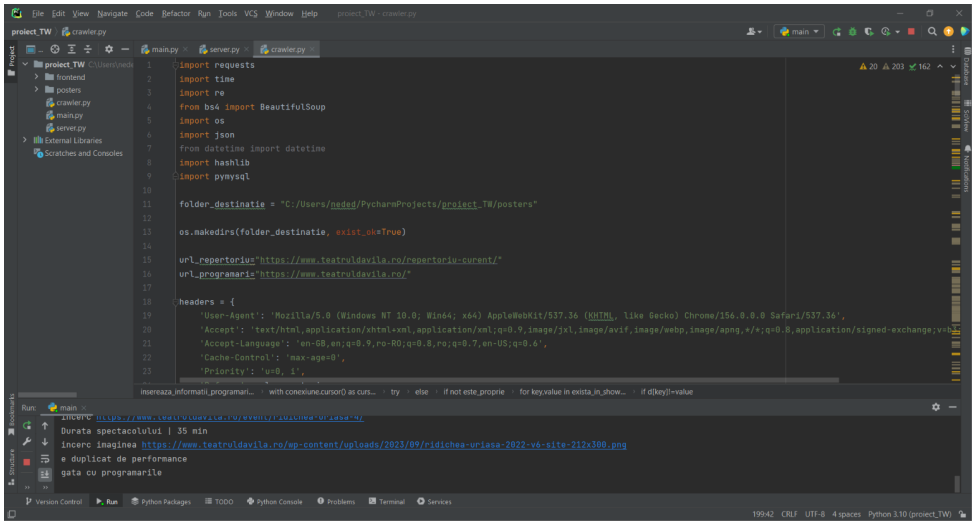


Figura 1: Organizarea fișierelor și directoarelor proiectului în mediul de dezvoltare

4.3 Rularea și oprirea aplicației

Sistemul utilizează o arhitectură bazată pe *multi-threading*, centralizată în scriptul `main.py`. Pentru a porni întregul ecosistem, se execută o singură comandă în terminal:

```
python main.py
```

La execuție, scriptul lansează simultan două fire de execuție :

1. **Crawler-ul de fundal :** Inițiază scanarea site-ului Teatrului Davila, descarcă afișele noi în folderul `posters/` și actualizează înregistrările din MySQL .
2. **Serverul web unificat :** Pornește pe adresa `http://127.0.0.1:5000`, expunând atât rutele API , cât și interfața grafică.

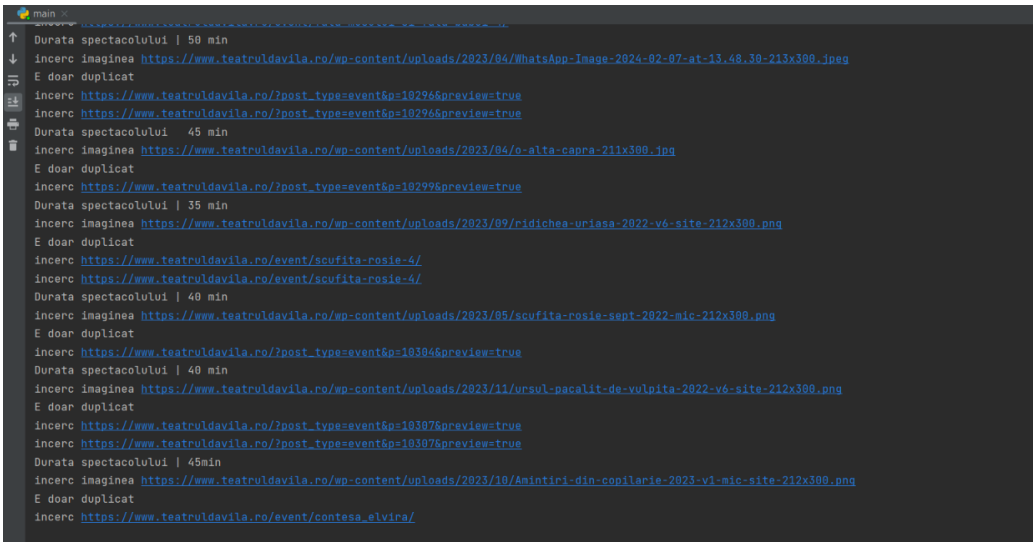


Figura 2: Execuția simultană a serverului Flask și a modulului crawler în terminal

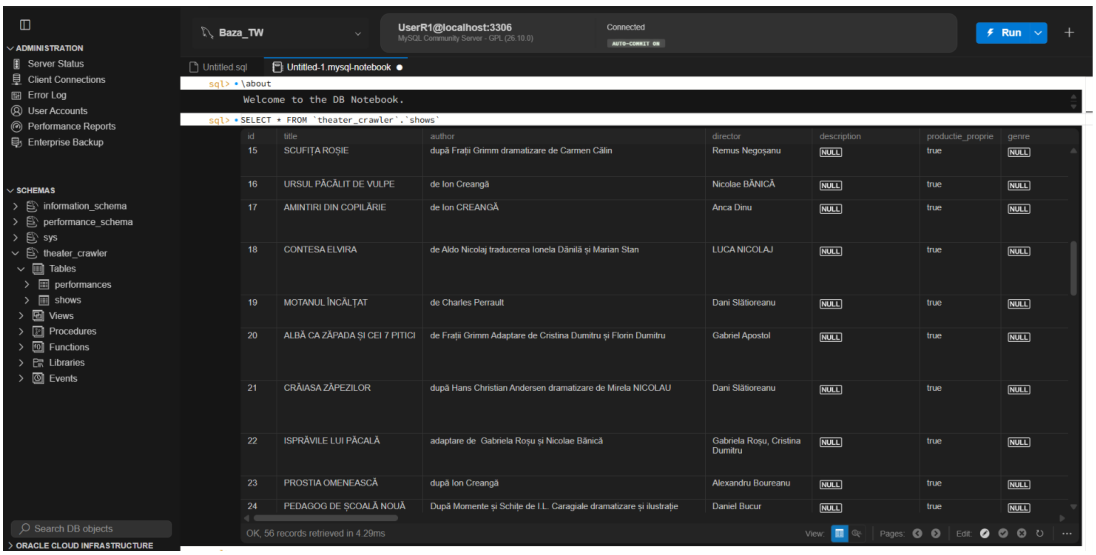


Figura 3: Persistența datelor extrase în tabela `shows` din baza de date MySQL

Oprirea sistemului: Deoarece ambele componente rulează ca procese *daemon*, închiderea aplicației se realizează prin apăsarea combinației de taste în terminal. Firul principal captează excepția `KeyboardInterrupt` și eliberează automat portul `5000` și conexiunile active.

4.4 Utilizarea interfeței web

După pornirea serverului, aplicația poate fi accesată din orice browser web la adresa `http://localhost:5000`. Utilizatorul are la dispoziție următoarele funcționalități:

- **Navigare SPA:** Accesarea rutelor `/`, `/shows`, `/shows/{id}` și `/schedule` fără reîncărcarea paginii, dar cu suport complet pentru butonul *Refresh* datorită rutei

catch-all din Flask .

- **Căutare și adaptabilitate mobilă:** Filtrarea instantanee a spectacolelor după titlu și adaptarea automată a grilei de afișare pe ecrane de dimensiuni reduse .

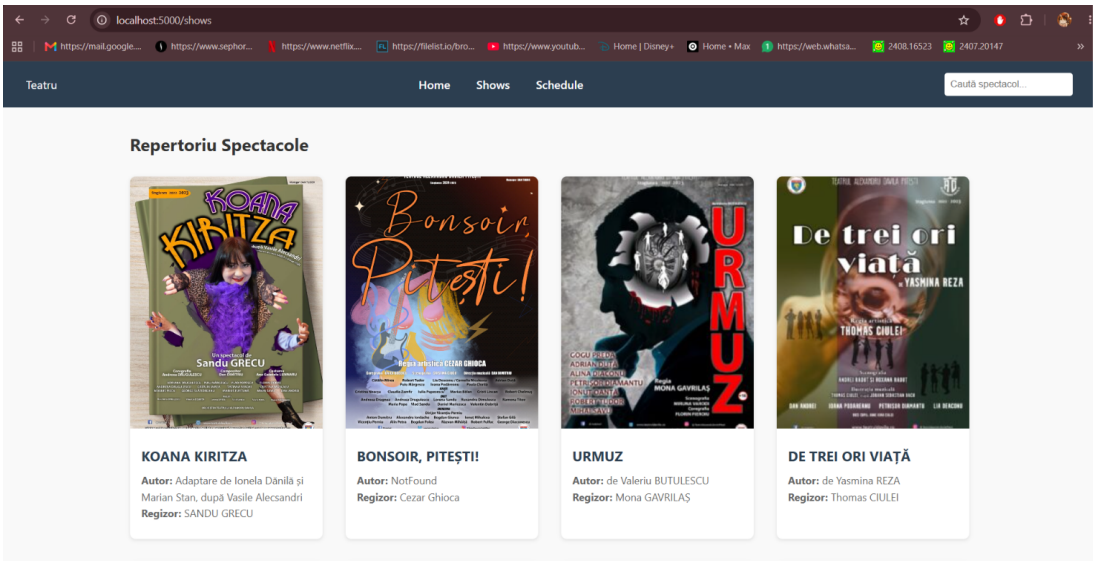
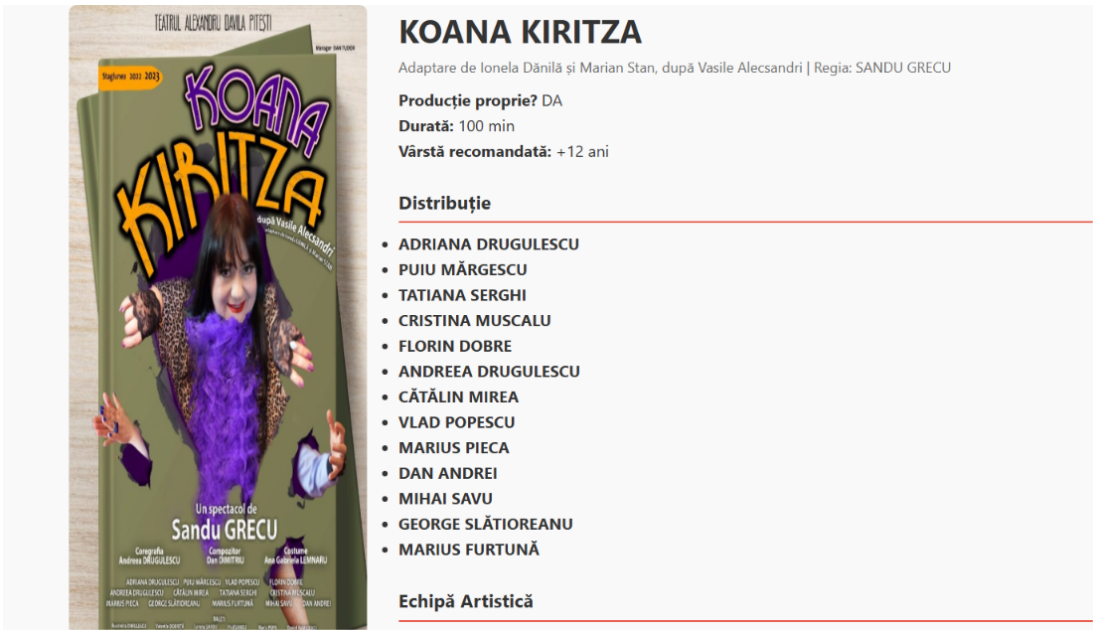


Figura 4: Pagina de repertoriu cu afișarea posterelor stocate local



S

Figura 5: Pagina individuală a unui spectacol cu distribuția și reprezentațiile viitoare

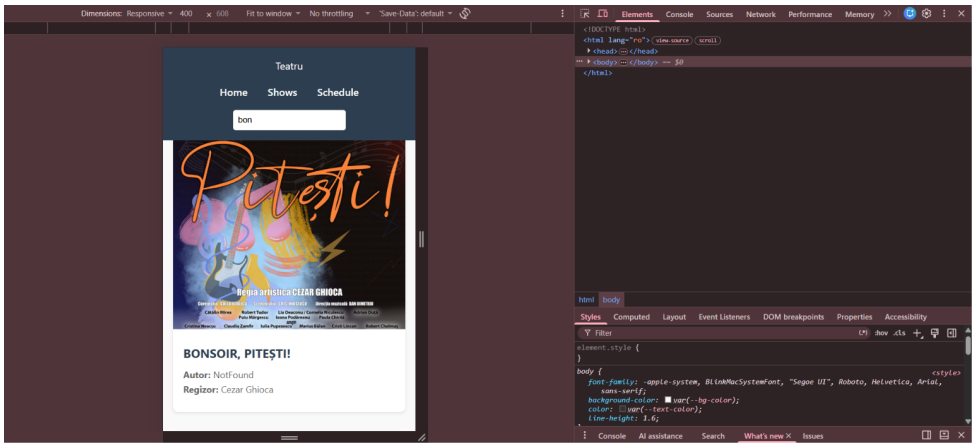


Figura 6: Adaptarea interfeței pe dispozitive mobile și funcționalitatea de căutare după titlu

5 Concluzii și perspective de dezvoltare

Proiectul de față a atins cu succes toate obiectivele propuse, concretizându-se într-o soluție software modernă și complet integrată pentru automatizarea preluării, stocării și afișării repertoriului Teatrului Davila.

Prin adoptarea unei arhitecturi hibride care combină un modul de crawling automatizat în fundal , o bază de date relațională MySQL și un server web unificat bazat pe Flask și Single Page Application, s-au obținut următoarele avantaje majore:

- **Autonomie și eficiență:** Eliminarea intervenției manuale pentru actualizarea programului spectacolelor, crawler-ul asigurând o sincronizare periodică cu platforma oficială.
- **Performanță și fluiditate:** Interfața SPA construită în Vanilla JavaScript oferă o experiență de navigare rapidă, fără timpi de reîncărcare a paginilor, completată de un sistem de rutare robust și de o căutare dinamică în timp real.
- **Independență media:** Descărcarea și stocarea locală a posterelor garantează disponibilitatea continuă a resurselor vizuale, chiar și în cazul unor modificări sau indisponibilități temporare ale site-ului sursă.
- **Optimizarea deployment-ului:** Unificarea serverului de frontend și a API-ului sub aceeași instanță Flask pe portul `5000` a simplificat considerabil configurarea, eliminând problemele de tip CORS și erorile de rutare la reîncărcarea paginilor.

5.1 Perspective de dezvoltare

Pentru viitor, aplicația poate fi extinsă prin implementarea unor module suplimentare care să sporească utilitatea și scalabilitatea acesteia:

- **Containerizare :** Împachetarea aplicației și a bazei de date în containere Docker separate, orchestrate prin `docker-compose`, pentru un deployment simplificat și portabilitate maximă pe servere dedicate sau platforme de tip cloud .
- **Sistem de autentificare și rezervări:** Extinderea bazei de date și a API-ului pentru a permite înregistrarea utilizatorilor, autentificarea securizată și rezervarea biletelor pentru reprezentațiile viitoare.
- **Monitorizare și telemetrie avansată:** Integrarea unor instrumente de observabilitate pentru a monitoriza în timp real starea crawler-ului, consumul de resurse și timpii de răspuns ai serverului Flask.

6 Bibliografie

1. **Python Software Foundation**, *Python Documentation: The Python Standard Library - threading, os, time*, disponibil online la: `https://docs.python.org/3/library/`.
2. **Pallets Projects**, *Flask Documentation: Quickstart, Routing, and Static Files*, disponibil online la: `https://flask.palletsprojects.com/`.
3. **Mozilla Developer Network** , *History API - Web APIs*, disponibil online la: `https://developer.mozilla.org/en-US/docs/Web/API/History_API`.
4. **Mozilla Developer Network** , *Fetch API - Using Fetch*, disponibil online la: `https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API`.
5. **PyMySQL Contributors**, *PyMySQL: Pure-Python MySQL client library*, disponibil online la: `https://pymysql.readthedocs.io/`.
6. **MySQL AB**, *MySQL 8.0 Reference Manual - Data Definition and Manipulation Statements*, disponibil online la: `https://dev.mysql.com/doc/refman/8.0/en/`.